FSD / FWP
Semester 1, 2026

Week 07

Dipto.Pratyaksa@rmit.edu.au

Database programming and ORM

Writing your own APIs and

Server-side using Node & Express.js;

RMIT UNIVERSITY

Segment 1

Remaining React Hooks

Database programming and ORM

Writing your own API and

Server-side using Node & Express

React Hooks

1. useState
2. useEffect
3. Custom hooks
4. useRef
5. useContext
6. useMemo
7. useCallback
8. useReducer
9. **useDeferredValue**
10. **useTransition**

useDefferedValue Hook

- Improves UI responsiveness by deferring the update of non-urgent, slow-rendering parts of the UI, allowing high-priority updates (like user input) to render immediately

- const deferredValue = useDeferredValue(value);
- During updates, React first renders with the old value, then schedules a background re-render with the new value.

- Use cases: filtering large lists, complex graphs, or data visualization based on input, where a delay in updating the results is acceptable.

Search-as-You-Type

```
import React, { useState, useDeferredValue } from 'react';

// Simulated large dataset for demonstration purposes
const largeDataset = Array.from({ length: 10000 }, (_, index) => `Item ${index + 1}`);

const SearchAsYouType = () => {
const [query, setQuery] = useState('');
// Deferring the search query to make the input more responsive
const deferredQuery = useDeferredValue(query);

const handleInputChange = (e) => {
setQuery(e.target.value);
};

// Simulated search function that filters the large dataset based on the deferred query
const filteredItems = largeDataset.filter((item) =>
item.toLowerCase().includes(deferredQuery.toLowerCase())
);
```

```
return (
<div>
<h2>Search-As-You-Type Example</h2>
<input
type="text"
value={query}
onChange={handleInputChange}
placeholder="Search items..."
style={{ padding: '10px', fontSize: '16px', width: '300px' }}
/>
<ul>
{filteredItems.slice(0, 20).map((item, index) => (
<li key={index}>{item}</li>
))}
</ul>
</div>
);
};

export default SearchAsYouType;
```

useDeferredValue – React

useTransition Hook

- Similar to useDefferedValue, it lets you render a part of the UI in the background
- useTransaction works with state change, useDefferedValue works with a received value
- It allows you to update component state without blocking the user interface
- It splits rendering into urgent updates (e.g., typing) and transition updates (e.g., rendering large lists), optimising performance
- Use cases:
- When a state change causes a heavy, slow re-render like filtering a large list or switching complex tabs.
- When you need to keep inputs responsive while data is loading

useTransition Hook

```
const [isPending, startTransition] = useTransition();

function handleTabChange(nextTab) {
  startTransition(() => {
    setTab(nextTab); // This update is marked as a transition
  });
}
```

useTransition – React

RMIT

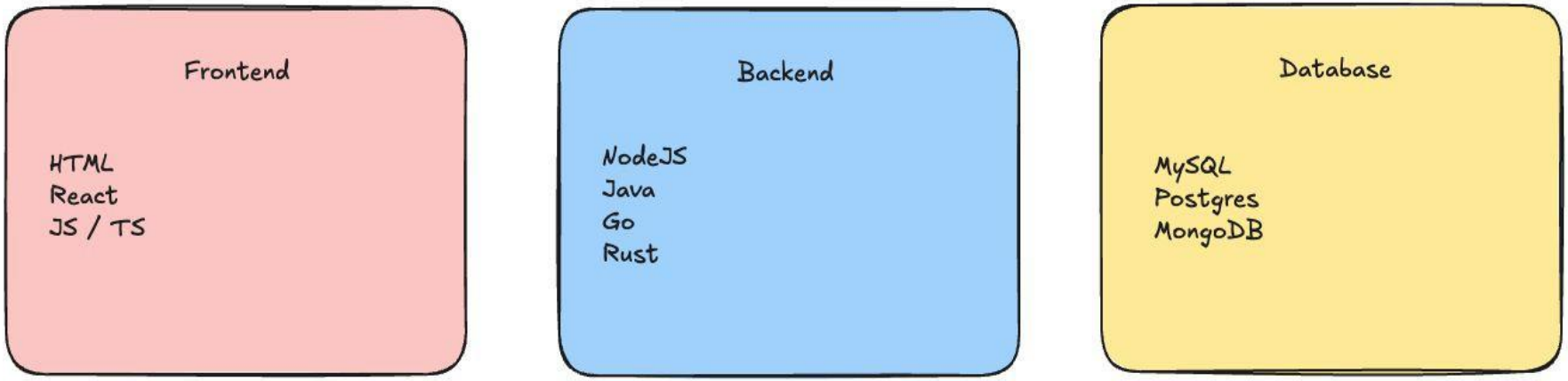
Database programming

- Consider a scenario where you want store the website/user data, *etc.* in a database instead of localStorage, files, etc.
- Unfortunately, React itself has no answer for that. It is a front-end framework.
- Next does support writing backend code but we won't use that
- This means you cannot connect React directly to any database say- MySQL, Postgres, MongoDB *etc.*
- *You will now need to connect React i.e. frontend to backend layer (responsible for talking to database)*
- This brings us to a notion of Full Stack Development!

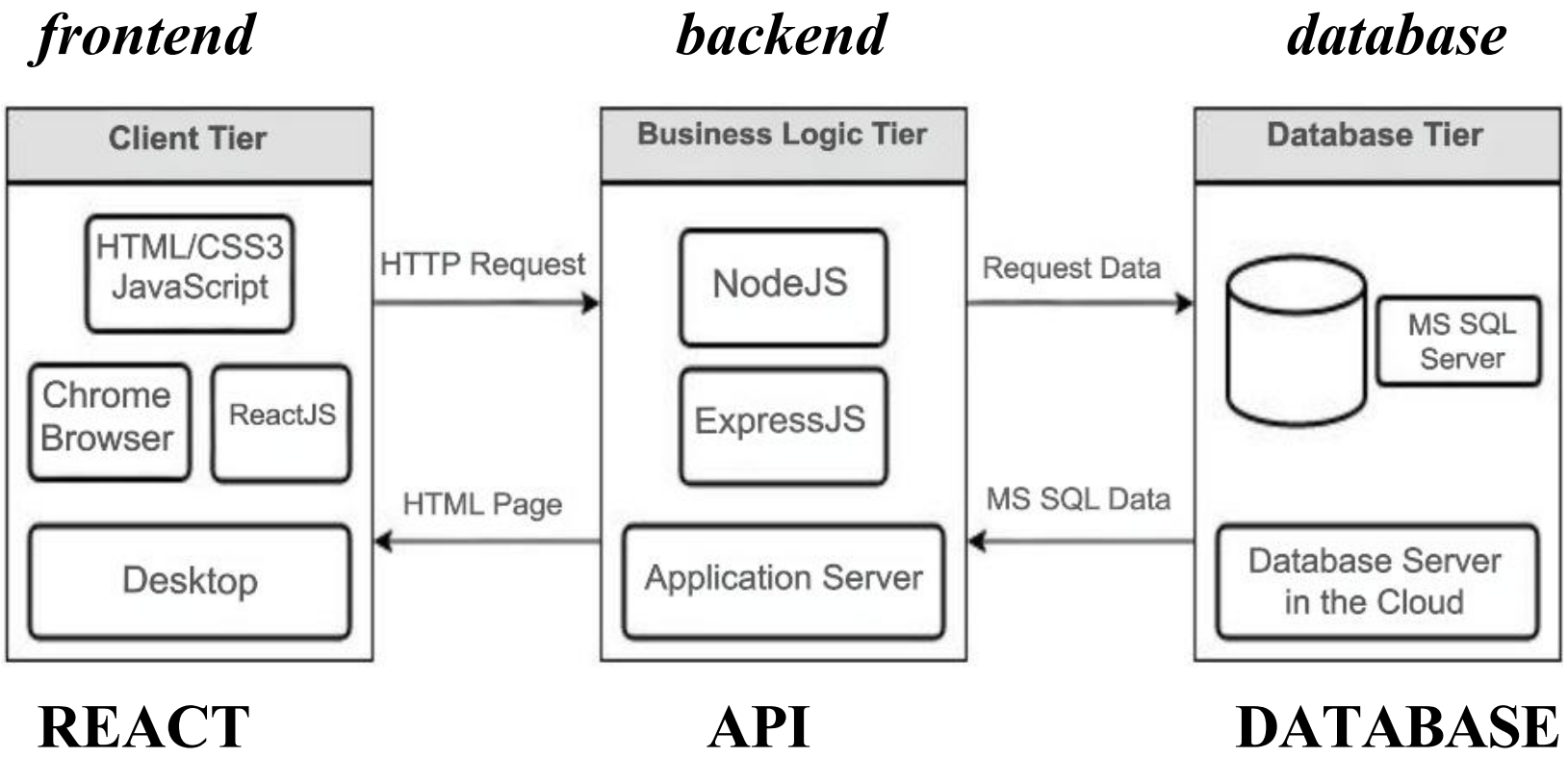
Full Stack Development (FSD)

- React is directly concerned with rendering and controlling UI elements.
- To interact with a database is by creating a database that can be retrieved through server-side language (such as PHP, Java, Node, or Python).
- The server-side language can receive requests for data from the React app.
- This is where an API comes into play.
  - The API will allow React to make requests to the server and receive data from the database.
  - You will have to write this API on your own.

FSD technologies



FSD *in this course*



For this course

- For the middle layer, we will use
  - Node.js: is an open source and cross-platform runtime environment for executing JavaScript / Typescript code outside of a browser i.e. on the server.
  - Express.js: is a small framework that sits on top of Node.js's web server functionality to simplify its APIs and add helpful new features- you will have to install it.

- For the backend database, we will use
  - Cloud MS SQL Server database
  - Your accounts have been created and details on Week 7 Prac sheet

Database programming via ORM

- In order to retrieve data from the database, one can always write good old SQL, but we will avoid it
- Instead, we will use an ORM known as 'TypeORM'- you will have to install it.
- TypeORM: TypeORM is a promise-based Node.js ORM for MySQL, Postgres, MariaDB, SQLite and Microsoft SQL Server.
  - It features solid transaction support, relations, eager and lazy loading, read replication and more.
  - [https://typeorm.io/]
- You can write database queries in JavaScript / Typescript!

What is an ORM?

- Object Relational Mapping
- An ORM software provides an object-oriented layer between relational databases and programming languages without having to write SQL queries.
- It standardises interfaces reducing boilerplate and speeding development time.
- Obvious advantage: you do not have to delve into pesky SQL syntax
  - ORM generates the SQL in the background
- *Can you think of disadvantages?*

Lectorial Exercise

- Why avoid SQL while writing an FSD application?
- Does ORM completely preclude the use of SQL?
- Do you know of any other widely used ORMs in the CSIT industry?

Database programming via ORM

- So essentially TypeORM will help us to write database queries in the middle layer i.e. Node.js, Express,js without having to write actual SQL
- We will write our queries in JS (Node.js being JavaScript / Typescript based)
- TypeORM will do the heavy-duty work of converting these to SQL in order for MySQL database to understand database operations.
- This means we will have to learn TypeORM syntax!
- Here is the official documentation
  - https://typeorm.io/

TypeORM Basics

- Time to get accustomed with TypeORM syntax / approach
- Some core concepts:
  - Models / Entities
  - Relationships

```
import {
  Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn,
} from "typeorm";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  lastName: string;

  @Column({ unique: true })
  email: string;

  @Column()
  age: number;
```

Node.js and Express.js?

- The Business Logic Tier will be written using Node.js and Express.js
- This tier represents the Application Server that will act as a bridge of communication for the Client Tier and Database Tier.
- This tier will serve HTML pages to the user's device and accept HTTP requests from the user and follow with the appropriate response.
- In our case, this means the API that we are going to write for database operations.
- The API will talk to database and return JSON
- React is able to process JSON.

Database programming: *my head hurts*

- So here is the gist of what we have talked about
  - Front end is React app which will no longer have to deal with code that deals with data
  - Backend uses Node and Express- we will write an API that talks to backend database and return data to front end. We will avoid writing SQL to talk to database, instead use an ORM known as *TypeORM*
  - Database is the Cloud MS SQL Server database.

Database programming: Full stack React app

- Let us build a full-stack React app
  - example 1
- Two separate projects
  - One project: node-express-typeorm ; an api to perform database operations
  - Second project: React app that consumes the API to interact with the database
    - It will use a library/module known as 'Axios' that will help React app call the API
    - Axios: make http requests from node.js
    - https://axios-http.com/docs/intro

What is this *Axios* thingie!!

- Axios is a promise-based HTTP Client for Node.js and the browser. It is isomorphic
  - it can run in the browser and Node.js with the same codebase.
  - [https://axios-http.com/docs/intro]

Lectorial Exercise

- What is a *promise*?

Example 1: step 1

- Create first project
- With Node and Express we will write
  - Code to create a database table without writing single line of SQL with the help of TypeORM to create a table say ‘user’ in the database
  - Then write an API with following methods:
    - Get all users
    - Get user based upon their id
    - Create new user
    - Edit a user and
    - Delete a user

Segment 2

Steps in creating a Full Stack App

Accessing your DB

Course admin:

- **Labs**
- **A2**

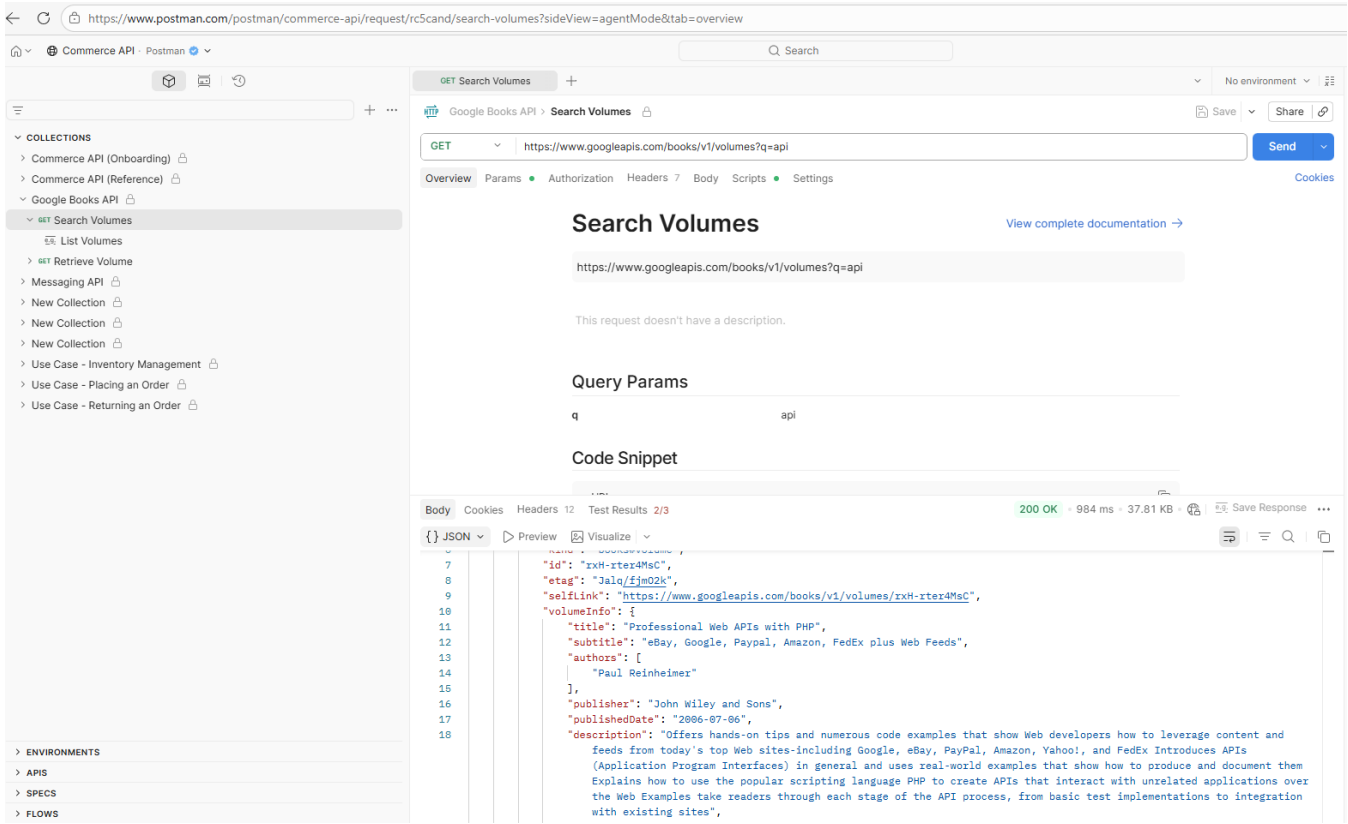
Steps involved

1. Create Node.js app
2. Install necessary modules: express, TypeORM, mssql
3. Setup Express web server so that API is available via a Url
4. Configure Cloud SQL database settings in the project so that TypeORM is aware of the location of actual database
5. Initialise TypeORM
6. Define TypeORM Entity
7. Write logic for API methods in controller
8. Define API routes
9. Run the express web server so that API is up and running
10. Test the API

How does one test an API?

- Usually, an API on its own solves no purpose
- Another app needs to call/consume it
- But there are software(s) available that will help you test API methods before you write a whole app to test the API methods
- POSTMAN is one such software
  - https://www.postman.com/

POSTMAN



Let us now use Postman to call the API endpoints!

Example 1: step 2

- Create second ie React project
- Make it aware of the API location
- Communicate with API using Axios
- Write components using hooks to call the methods of the API to interact with the database table ‘tutorial’

Steps involved

1. Create new React app
2. Create sub-directories for components, services
3. Create .env file in root folder
4. Modify .tsx files accordingly
5. Write code for components
6. Write code for services
7. Make aware of the React app where API is running
8. Run the React App
9. Perform CRUD (Create- Update- Delete) operations!

Lab # 7

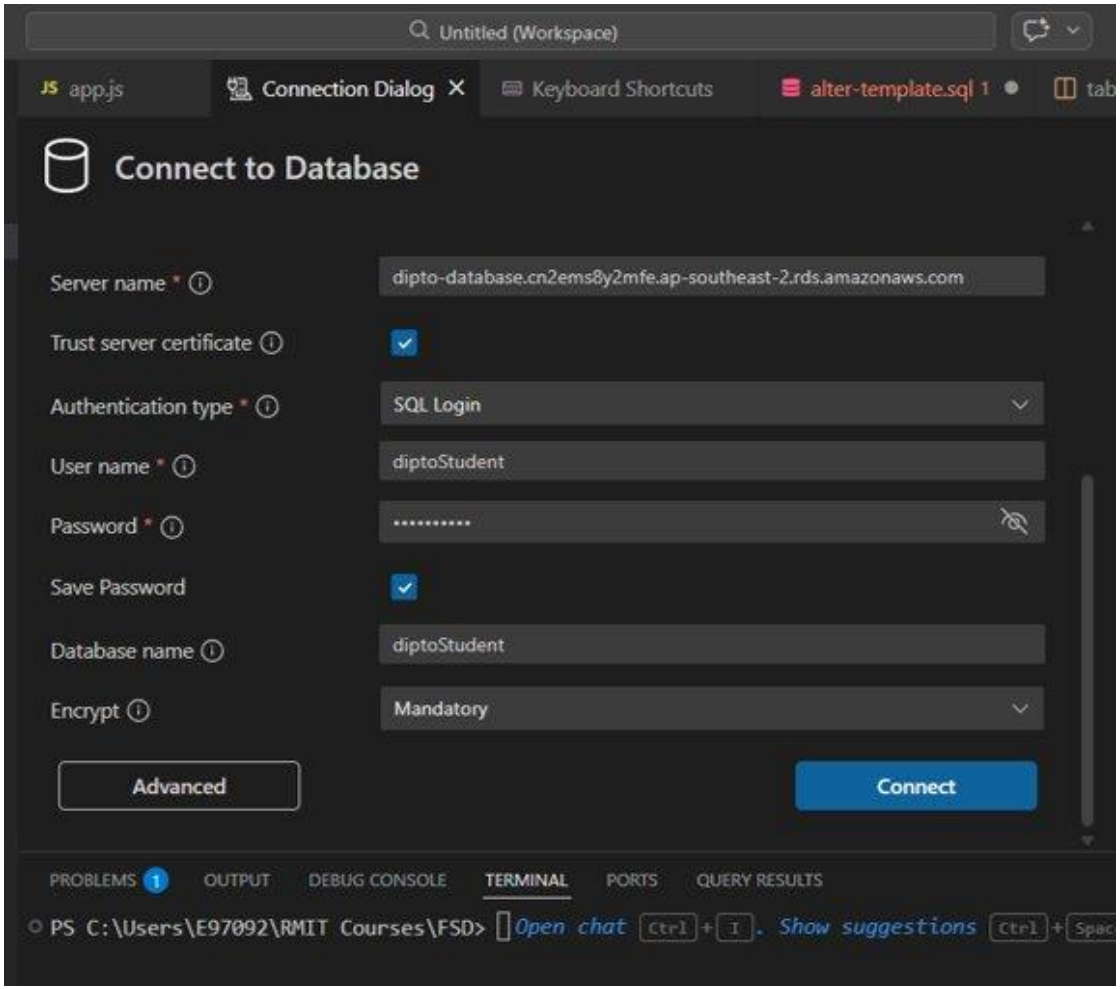
- As you can see that full stack development is not a brief process
- You need to follow a lot of steps

- Remaining React Hooks
- Accessing DB account on AWS RDS.
  - Login details supplied on the prac sheet

Accessing DB server

Using VS Code mssql extension:

Connect to DB

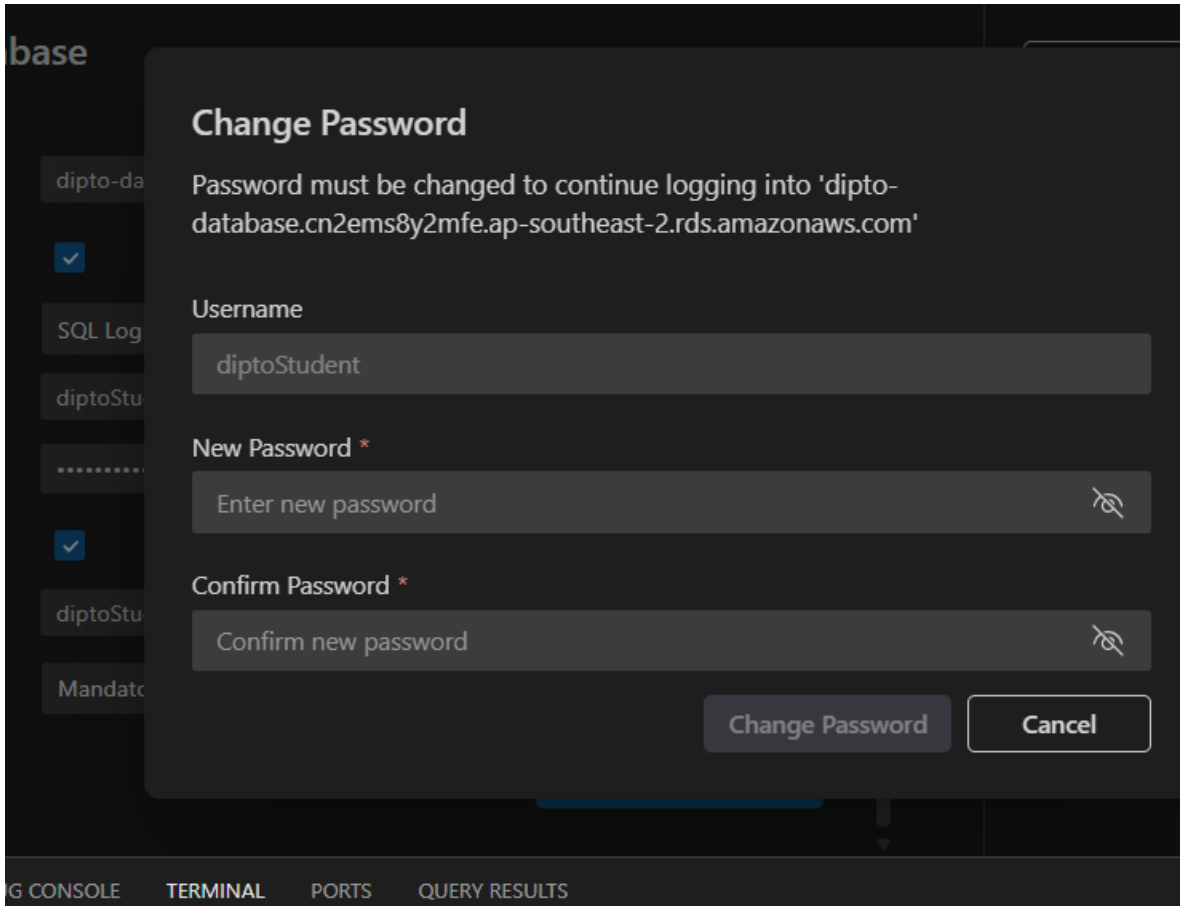


RMIT

Accessing DB server

Using VS Code mssql extension:

Change the default password
once

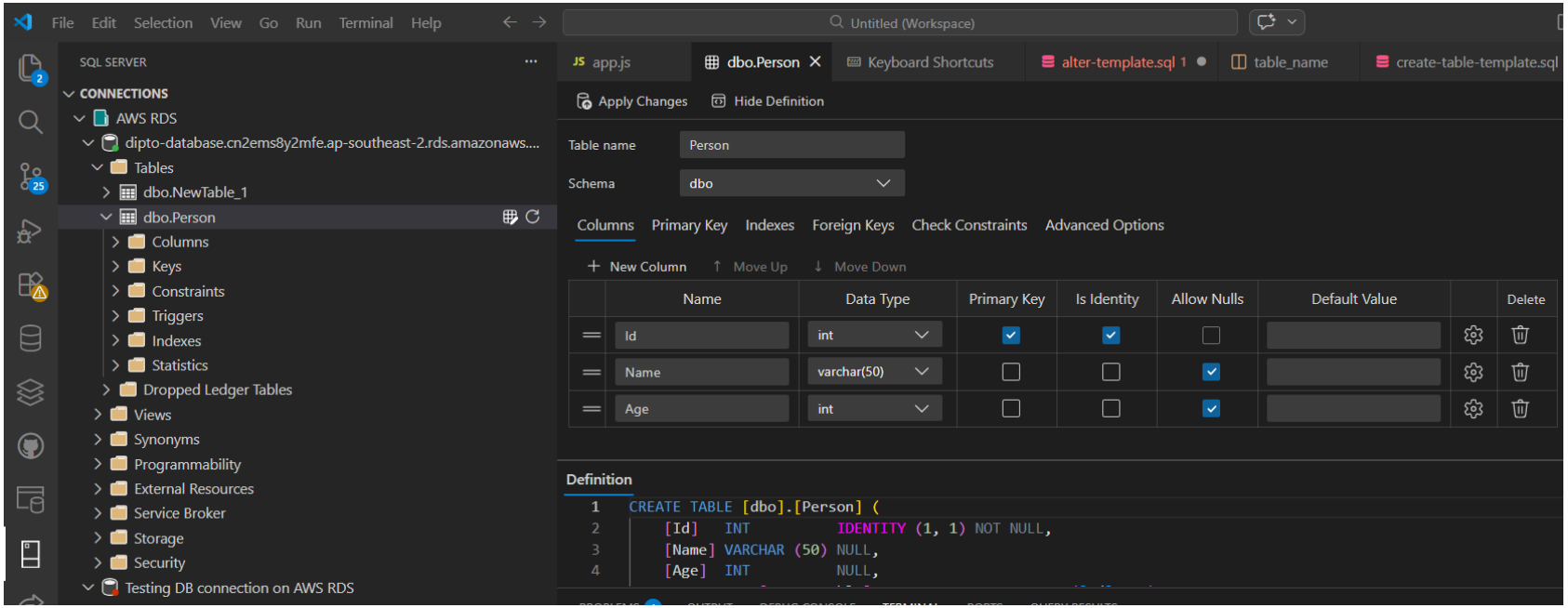


RMIT

Accessing DB server

Using VS Code mssql extension:

Start getting familiar with the interface



RMIT

Lab # 8

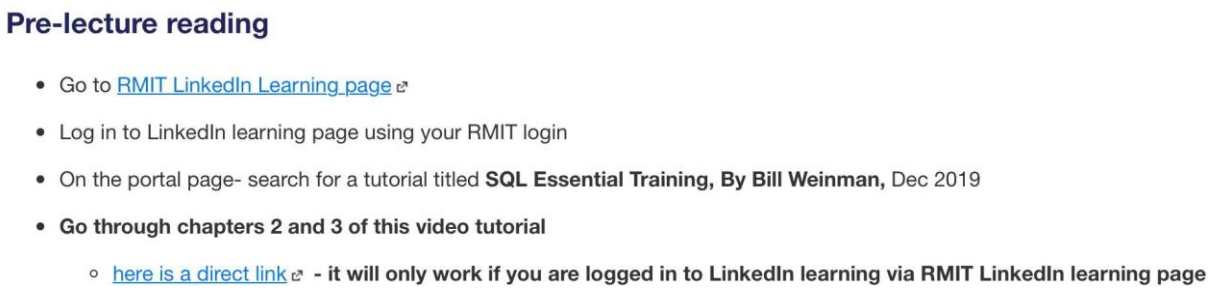
- You will create Full Stack App as a lab exercise next week
- You will also learn to do the login-logout-registration example with database instead of local Storage next week.
- So do not miss next week's lab ie lab 8 as it will form the basis of almost everything you will do for Assignment 2.

Assignment 2

- Will be published on Canvas as of Friday this week
- Full stack web application
- To be completed in pairs
- Worth: 45%
- Requires use of
  - React, (Node, Express) & Cloud MS SQL Server database
  - GitHub on a regular basis

Before you start coding on assignment 2

- Take a breather and revise your database skills
- Read about ER diagrams
- Relations, keys, constraints
- Some reference in the pre-reading material for week 7 on Canvas



Pre-lecture reading

- Go to RMIT LinkedIn Learning page
- Log in to LinkedIn learning page using your RMIT login
- On the portal page- search for a tutorial titled **SQL Essential Training, By Bill Weinman,** Dec 2019
- **Go through chapters 2 and 3 of this video tutorial**
  - here is a direct link **- it will only work if you are logged in to LinkedIn learning via RMIT LinkedIn learning page**

- Draw ERD (Entity Relationship Diagram) for assignment 2, first!

References

- Reference: *The road to react (2025 edition), by Robin Weiruch; Leanpub*
- https://reactjs.org/docs/hooks-reference.html
- Understanding useDeferredValue in React: Enhancing Performance with Deferred Rendering | by Frontend Highlights | Medium [https://medium.com/@ignatovich.dm/understanding-usedeferredvalue-in-react-enhancing-performance-with-deferred-rendering-ec8eb28aa997]

Next week

- More on
  - Database programming and ORM
  - More on API
- Authentication in a React app
  - Writing your own
  - Using third-party authentication